

Please note that these are sample questions and do not impose any constraints on the questions that will appear on the actual final exam. All questions are based on the lecture contents. Sample solutions are NOT available.

Part A – Multiple Choice Questions

Question 1 through question 10 are multiple-choice questions. For each of the multiple-choice questions, only one answer is correct. Clearly state the question number and your answer in your answer sheets.

Question 1 (1.0 mark)

Given the following algorithm that takes in as parameters an integer array A and an integer number n :

Algorithm: myAlgorithm($A[]$: integer, n : integer)

```
int result = 0;
int i, j
for i = 0 to  $\frac{n}{2}$  do
    j = 0
    while  $A[j] < 0$  and  $j < \frac{n}{2}$  do
        result = result +  $A[i] \times A[j]$ 
        increment j by 1
    end while
end for
return result
```

What is the best and worst case asymptotic time complexity of the algorithm?

- A. Best case $O(n)$, worst case $O(n)$.
- B. Best case $O(1)$, worst case $O(n)$.
- C. Best case $O(n)$, worst case $O(n^2)$.
- D. Best case $O(\log_2 n)$, worst case $O(\log_2 n)$.
- E. Best case $O(1)$, worst case $O(n^2)$.

Question 2 (1.0 mark)

$\log(n!)$ is in all of the following sets, except

- A. $O(n^2)$
- B. $\Theta(n \log n)$
- C. $\Omega(\log n)$
- D. $O(n^3)$
- E. $O(n)$

Question 3 (1.0 mark)

Which one of the following statements is correct about "tail recursion"?

- A. Tail recursion should be avoided because programs with tail recursion can always be replaced with loops which are typically easier to implement and written.
- B. Tail recursion should be avoided because computer operating system requires space on its runtime stack for each function call, and because tail recursion can be replaced by a loop that needs no extra space.
- C. Tail recursion should be used whenever it is possible, because functions written in tail recursion can execute faster.
- D. Tail recursion should be used whenever it is possible, because the functions are easier to read than corresponding functions without loops and tail recursion functions use less memory space.
- E. None of the above.

Question 4 (1.0 mark)

The recurrence relation $T(n) = 6T(\lfloor \frac{n}{3} \rfloor) + n^2 \lg n$ can be tested using Master Theorem using

- A. Case 1: If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- B. Case 2: If $f(n) = \Theta(n^{\log_b a})$, then, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
- C. Case 3: If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$, and if $af(\frac{n}{b}) \leq cf(n)$ for some constant $c < 1$, and all sufficiently large n , then $T(n) = \Theta(f(n))$.
- D. Both Case 1 and Case 2.
- E. None of the above.

Question 5 (1.0 mark)

Why is it common for a circular queue implementation to waste one table element?

- A. To perform some loops faster
- B. To have a place to store the tail value
- C. To avoid confusing an empty queue with a full queue
- D. To make sure the queue always has at least one element in use
- E. To store the number of elements currently in the queue

Question 6 (1.0 mark)

Which one of the following statements is correct about linked lists?

- A. When creating a linked list, you need to allocate more memory than you initially need to store the head and tail pointers.
- B. Linked lists are taking up less memory per element as compared to an array.
- C. When the position where an element is to be added to or deleted from linked lists is given, adding and deleting elements into linked list become slower than array at adding and deleting elements because the algorithm needs to traverse the list to the indicated position.
- D. When the position where an element is to be added to or deleted from linked lists is given, adding and deleting elements into linked list become faster than array at adding and deleting elements because the algorithm does not need to traverse the list to the indicated position.
- E. Linked lists have the same run-time efficiency as array when adding elements into and deleting elements from array.

Question 7 (1.0 mark)

Consider the *QUICKSORT*($A, low, high$) algorithm that chooses the pivot value using middle-of-three technique. The recursive calls *QUICKSORT*($A, low, mid - 1$) and *QUICKSORT*($A, mid + 1, high$). Let $low = 1$ and $high = 9$. Using the left-guard and right-guard partition algorithm, as discussed in lecture and middle-of-three technique of selecting a pivot value, what is the pivot value and where is the position in the array the pivot value will be placed. Assume the initial state of the array $A[low, high]$ looks as follows:

0	1	2	3	4	5	6	7	8
9	3	7	11	5	1	8	12	6

- A. Pivot: 9, position placed in array: 6
- B. Pivot: 6, position placed in array: 3
- C. Pivot: 5, position placed in array: 2
- D. Pivot: 7, position placed in array: 4
- E. Pivot: 8, position placed in array: 5

Question 8 (1.0 mark)

Given an array A storing n elements. Which one of the following statements is true?

- A. Quicksort is always the fastest algorithm in practice to sort A .
- B. Shell sort is always the fastest algorithm in practice to sort A .
- C. If A is almost sorted, selection sort should be used.
- D. If A is so large that it does not fit into the main memory, merge sort should be used.
- E. If A is so large that it does not fit into the main memory, insertion sort should be used.

Question 9 (1.0 mark)

Dijkstra's algorithm computes:

- A. All-pairs shortest paths
- B. The minimum spanning tree
- C. Hamiltonian path
- D. Guaranteed shortest path
- E. The shortest-path from a single source

Question 10 (1.0 mark)

The fractional knapsack problem is an example of

- A. Dynamic programming
- B. Branch-and-Bound algorithm
- C. Bread-first search
- D. Greedy technique yielding an approximate solution
- E. Greedy technique yielding an optimal solution

Part B – Application of Concepts

Algorithm Analysis and Algorithm Complexity

Question 11 (Total 10 Marks)

- a) Suppose we are comparing two similar algorithms. Suppose that for all inputs of size n , the first algorithm run in $5n^2$ steps per second, while the second algorithm runs in 2^n steps per seconds. For which values of n does the second algorithm run faster than the first algorithm? **(2.0 marks)**
- b) Is it true that $24^n \in O(8^n)$? Justify your answer using asymptotic analysis. Answer without a proper explanation will have mark deducted. **(2.0 marks)**
- c) Given the following recursively implemented sorting algorithm, $RecSort(int[] A, int n)$, which takes as input an unsorted array A of n integers and returns an array with the elements of the array A sorted in ascending order. The pseudocode of the $RecSort(int[] A, int n)$ is as follow:

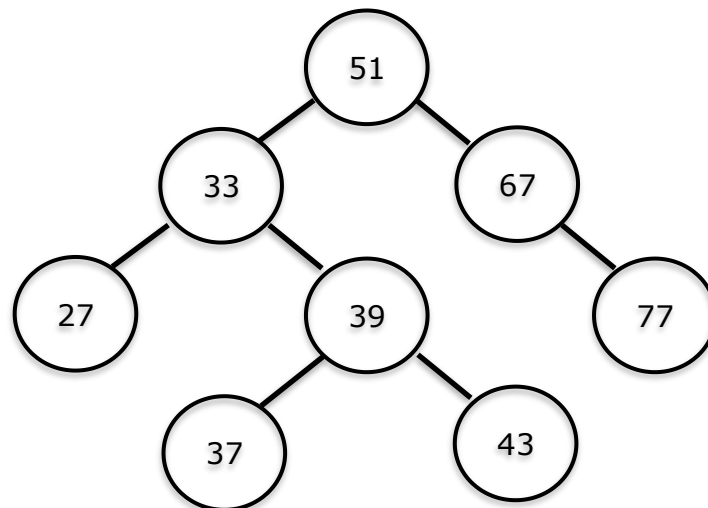
```

RecSort(int [] A, int n)
if n = 1 then
    Return A[0]
m = A[0]
index = 0
for i = 1 to n - 1
    if A[i] < m then
        m = A[i]
        index = i
    swap A[0] and A[index]
return A[0] × RecSort(A[1..n], n - 1)
    
```

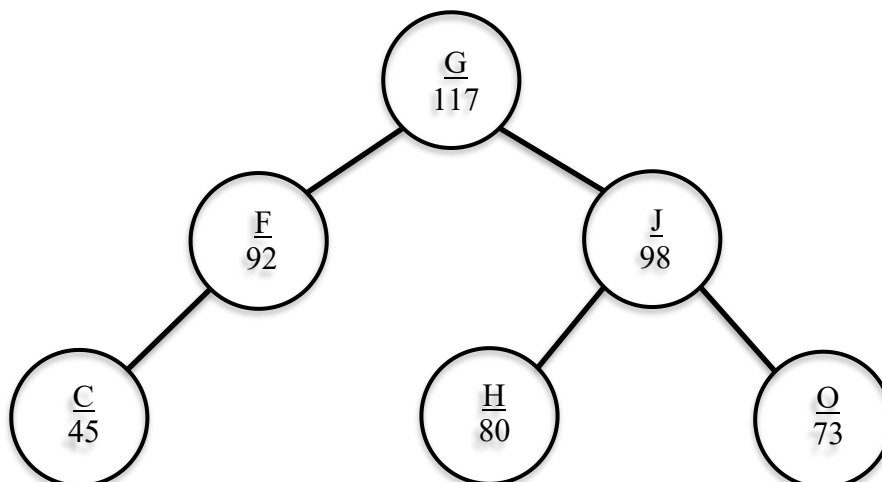
- (i) Let $T(n)$ be the running time complexity of the $RecSort(int[] A, int n)$ algorithm, write the recurrence equation that describes the running time complexity. **(1.0 mark)**
- (ii) Solve the recurrence equation $T(n)$ to determine the upper- and lower- bound complexity of the $RecSort(int[] A, int n)$ algorithm. **(2.0 marks)**
- d) Solve the following recurrence equations. For all the recurrence equations, $T(1) = 1$. You may use Master Theorem or expansion and substitution techniques.
- (i) $T(n) = 2T\left(\frac{n}{4}\right) + 1$ **(1.0 mark)**
- (ii) $T(n) = 2T\left(\frac{n}{2}\right) + n^3$ **(1.0 mark)**
- (iii) $T(n) = 4T\left(\frac{n}{2}\right) + n^2$ **(1.0 mark)**

Data Structure**Question 12 (Total 10 Marks)**

- a) Write an algorithm, *NotHereAndThere*(*int[] A*, *int[] B*), that takes as input two integer arrays, array *A* of size *n* and array *B* of size *m*. Both arrays *A* and *B* store integer numbers that are in increasing orders that may or may not be the same. You can assume that both arrays *A* and *B* has no repeating numbers, but the numbers that appear in *A* may appear in *B* and vice versa. Your algorithm should print all the numbers that are not in array *A* and not in Array *B*, count the total number of elements of array *A* that do not occur in array *B* plus the number of elements of array *B* that do not occur in array *A* and print the total count. For example, if array *A* = {3,4,7,9,11} and array *B* = {1,4,6,7,11}, your algorithm should return 4 as the total number of elements that are not in array *A* and array *B*, and the numbers are 1,3,6,9. The runtime complexity of your algorithm should be in the order of $n + m$, that is, $O(n + m)$. Note: You are not allowed to declare a third array, but you are allowed to declare any necessary counters (variables.) **(3.0 marks)**
- b) Draw an AVL tree resulting from the insertion of an entry with key 41 into the AVL tree shown here. Show each step and any rotation performed. **(1.0 mark)**



- c) Insert into the following TREAP an identifier 'N' that has a priority of 131. Show each step and any rotation performed. **(2.0 marks)**

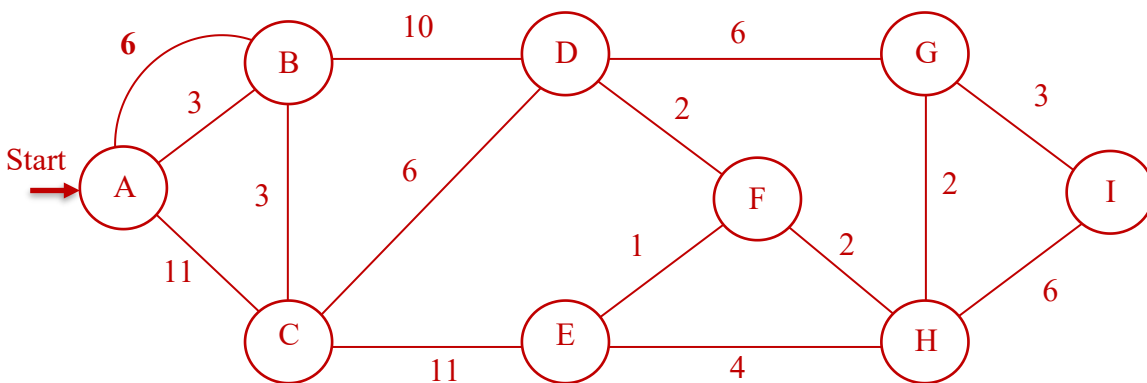


- d) Consider a hash table of size 7 with hash function $h(x) = x \bmod 7$. Draw the table that results after inserting, in the given order, the following values: 237, 428, 261, 374, 783, 346, 409 for each of the TWO scenarios below:
- When collisions are handled by separate chaining; **(2.0 marks)**
 - When collisions are handled by double hashing using a second hash function $h'(x) = (2x - 1) \bmod 5 + (1)$. Hint, the overall (combined) hash function is $H(x) = (h(x) + i \times h'(x)) \bmod 7$, where $i = 0, 1, 2, 3, \dots$. **(2.0 marks)**

Graph and Greedy Algorithms

Question 13 (Total 10 Marks)

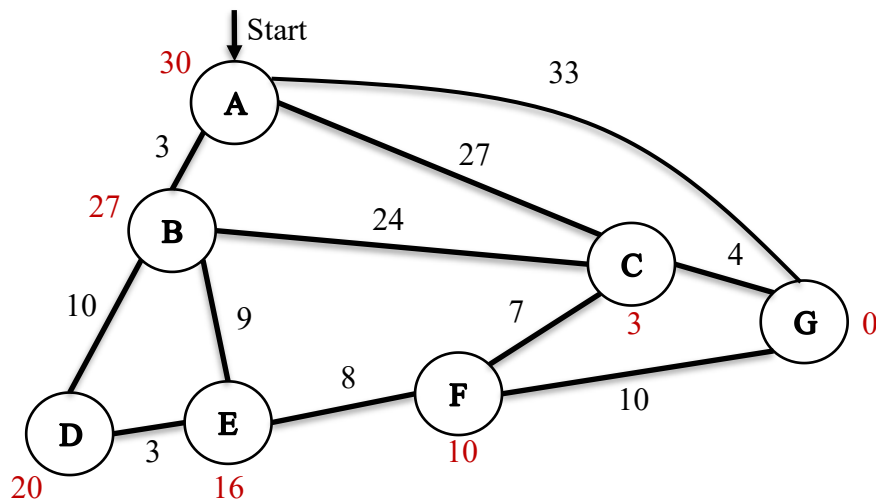
- a) Given the following undirected graph:



Represent the graph as:

- Adjacency matrix **(1.0 mark)**
 - Adjacency list **(1.0 mark)**
 - Incidence matrix **(1.0 mark)**
- b) Given the above-undirected graph, use Dijkstra's shortest-path algorithm to compute the shortest path from A to all network nodes.
- Show how the algorithm works by computing a table like the one discussed in class. **(3.0 marks)**
 - Show all the paths from node A to all other network nodes. **(1.0 mark)**

- c) Consider the following search problem, represented as a graph. Each node is label by a capital letter and the value of a heuristic function is shown in maroon. Each edge is labelled by the cost to traverse that edge. The start state is 'A' and the only goal state is 'G'.
- (i) Is the heuristics specified in the problem above permissible? Justify your answer. If the heuristics is permissible, proceed to answer part c(ii), if the heuristics is not permissible, correct it with a sensible value of your choice and proceed to answer part c(ii). **(1.0 mark)**
- (ii) Perform the A* search to find the shortest path from node A to node G. **(2.0 marks)**



Numbers in maroon are the heuristics for the respective nodes.
Numbers in black are the weight (estimated costs.)

End of Examination